The BTRON BX Series: A Family of Clean FPGA RTL Machines with Scalable Vector Extensions from 16-bit to 64-bit

Independent Systems Architecture Study
Umbrella Technical Note — `btron-bx-series`

November 2026

Abstract

The BTRON BX series is a coherent family of clean, synthesizable FPGA RTL machines that revive the peripheral ecosystems of classic 8-bit through 64-bit platforms on a single open ISA. Every host is RISC-V. The 8-bit and 16-bit members are not new base ISAs: they are RV32I harts whose *datapath and data bus* are 8 or 16 bits wide, executing the standard 32-bit instruction encodings over multiple cycles. Their vector unit is SVM (Scalar–Vector–Matrix), a named downward profile of the RISC-V Vector Extension (RVV 1.0). BX-32 and BX-64 leave the profile and implement ratified subsets (Zve32f, then Zve64d or full V). All designs are expressed in Bluespec SystemVerilog, target contemporary FPGAs, and share one stance: respect for the original engineering, clean architecture, and open accessibility.

1 Motivation and Unifying Philosophy

Classic home-computer and console architectures solved difficult real-time media problems under severe cost, power and transistor constraints. Their peripheral sets and specialised compute units remain instructive. The BTRON BX programme does not attempt commercial competition or cycle-accurate commercial emulation. Instead it asks:

> Can the essential organisation of these machines—their peripheral libraries, their dual-processor or vector-unit structure, their DMA and local-memory philosophy—be re-expressed in clean, modern, synthesizable RTL that any competent engineer can read, modify and run on an FPGA?

From BX-16 upward an additional requirement is imposed: the vector capability itself must be *scalable*. The same programming model, the same configuration mechanism, and the same element-type hierarchy should appear at every width, merely with larger minimum VLEN, wider supported element widths, and richer floating-point support as one moves from 16-bit to 64-bit hosts.

2 SVM — a Scalable Profile of RVV

SVM (Scalar–Vector–Matrix) is the name of the BX vector profile. It is not a private opcode map. Scalar instructions are RV32I (BX-8, BX-16) or RV32/RV64 (BX-32, BX-64). Vector instructions use the standard RVV OP-V encoding, the 32 architectural vector registers `v0`–`v31`, and `vsetvl`. Matrix and geometry operations that RVV does not define (`VMAT4VEC`, `VCROSS`) are a separate custom extension, `Xsvm`, and are never required for the SVM profile itself.

There is no ratified RISC-V base with XLEN of 8 or 16. An “8-bit RISC-V” or “16-bit RISC-V” in this series means an RV32I hart with a narrow datapath. XLEN stays 32 so that a `vsetvli` encoded for SVM-8 is still a legal RVV configuration instruction when the same program is moved to BX-32.

2.1 Profiles

BX	Datapath	SVM / RVV profile	What is implemented
BX-8	8-bit	SVM-8	ELEN 8, VLEN 32, EEW 8 only, integer
BX-16	16-bit	SVM-16	ELEN 16, VLEN 128, EEW 8/16, saturating
BX-32	32-bit	Zve32f	EEW 8/16/32, FP32, VLEN $\geq$ 128
BX-64	64-bit	V / Zve64d	EEW 8/16/32/64, FP32+FP64, VLEN $\geq$ 128

SVM-8 is *not* `Zve32x`: that subset requires EEW of 8, 16, and 32. SVM-16 is not `Zve32x` either: its ELEN is 16, so a `vsetvl` that asks for SEW=32 is illegal. Both profiles are teaching subsets of the RVV instruction set, implemented with the real encoding, and identified by a custom `misa`-adjacent capability bit rather than by claiming a ratified Zve string. From BX-32 upward the claim becomes a real extension string.

Q16.16 is not an architectural element type. On SVM-16 a 16-bit lane multiply widens into a 32-bit result (`vwmul`); the shift that builds a Q16.16 value is ordinary arithmetic. The old 16-register, 16-bit-opcode VEX coprocessor is withdrawn. It could not be RVV.

3 Scalable Vector Architecture across the Series

The vector story is deliberately aligned with the ratified RISC-V Vector Extension and its embedded subsets, because those subsets already encode a clean scaling path:

Subset	Min VLEN	EEW	FP
Zve32x	32	8, 16, 32	none
Zve32f	32	8, 16, 32	FP32
Zve64x	64	8, 16, 32, 64	none
Zve64f	64	8, 16, 32, 64	FP32
Zve64d	64	8, 16, 32, 64	FP32+FP64
V (full)	128	8, 16, 32, 64	FP32+FP64

The BX series maps onto this hierarchy exactly as shown in the preceding table.

3.1 BX-16 — First Appearance of Scalable Vectors

BX-16 introduces a lightweight, integer-centric vector unit whose programming model is source-compatible with the later RVV subsets:

- supported element widths SEW = 8 and 16 (native to a 16-bit host),
- modest VLEN (32 or 64 bits is sufficient; 128-bit registers are used),
- saturating arithmetic, widening multiplies and simple reductions—exactly the operations useful for Amiga-style blitter work, audio mixing and early OpenGL ES-style transform kernels,
- the same `vsetvl`-style configuration registers and length-agnostic loops that appear in BX-32 and BX-64.

Whether the host is a clean 16-bit proprietary core (R1600) or a soft 68000-class processor, the vector unit is attached as a coprocessor whose instruction encodings and register file can be carried forward unchanged when the design is later retargeted to RV32.

3.2 BX-32 — Practical Embedded Vector (Zve32f / Zve64f class)

BX-32 adopts a 32-bit RISC-V host (RV32) together with a vector unit belonging to the Zve32f or Zve64f family:

- VLEN $\geq$ 64 (commonly 128),

- SEW = 8, 16, 32,
- single-precision floating-point vector arithmetic (essential for Dreamcast-style matrix work and for OpenGL ES vertex processing),
- the same length-agnostic loops written for BX-16 continue to run, now with wider elements and floating-point support.

This configuration is sufficient to express the dual-SH-2 coordination patterns of the Saturn and the 4-wide floating-point matrix pipelines of the Dreamcast SH-4.

3.3 BX-64 — Full Application Vector (V / Zve64d class)

BX-64 employs dual 64-bit RISC-V cores, each equipped with a full RVV (or at least Zve64d) vector unit:

- VLEN $\geq$ 128 (256 or 512 preferred),
- SEW = 8, 16, 32, 64,
- single- and double-precision floating-point,
- the complete set of widening, narrowing, masked and reduction operations required to cover the Emotion Engine VU0/VU1 and the N64 RSP workloads.

Software written against the BX-16 or BX-32 vector model remains source-compatible; only the available element widths and the concrete VLMAX change.

4 The Four Machines (Summary)

- **bx-8** — MSX / R800 class. No vector unit; pure 8-bit foundation for the common BSV infrastructure and peripheral-library discipline.
- **bx-16** — 16-bit MSX / Amiga class + first scalable vector unit (R1600 + native VEX, SEW 8/16, modest VLEN, integer + saturating arithmetic). OpenGL ES path begins here.
- **bx-32** — Sega Saturn / Dreamcast peripherals on RV32 + Zve32f/Zve64f-class vector unit. Dual-core or single high-performance configurations. Full OpenGL ES vertex pipeline.
- **bx-64** — PS2 / N64 peripherals on dual RV64 cores + full RVV / Zve64d vector units. Highest throughput member of the family.

5 Common Design Principles

All four machines share the following invariants:

1. **Clean RTL.** Source is written in Bluespec SystemVerilog.
2. **Selectable peripheral libraries.** Parameterised modules preserve authenticity while allowing experimental configurations.
3. **Open or clean ISAs.** Proprietary cores are replaced by open standards or cleanly designed alternatives.
4. **Scalable vector model (from BX-16).** The same configuration mechanism, the same element-type hierarchy and the same length-agnostic programming style appear at every width (VEX $\rightarrow$ RVV).
5. **FPGA-first implementation.** Every design is synthesizable for contemporary FPGAs.
6. **Professional respect.** The series is framed as an architectural study that honours the original engineers.

6 Progression and Educational Value

Machine	Width	Vector capability
bx-8	8-bit	none (infrastructure only)
bx-16	16-bit	native VEX (SEW 8/16, Q16.16, 4-wide)
bx-32	32-bit	Zve32f/Zve64f-class, FP32, VLEN $\geq$ 64
bx-64	64-bit	full RVV / Zve64d, FP32+FP64, VLEN $\geq$ 128

A student or engineer can begin with bx-8, master the common BSV infrastructure, then acquire the scalable vector programming model on bx-16 and simply grow the same code to the richer element widths and floating-point support of bx-32 and bx-64.

7 Relationship to Companion Notes

- `btron-bx8.tex` — BX-8 (MSX / R800) detailed implementation note.
- `btron-bx16.tex` — BX-16 / BX-8 3D: R1600 core and the concrete VEX definition.
- `btron-bx32.tex` — Saturn / Dreamcast revival with 32-bit RISC-V + vector extension.
- `btron-bx64.tex` — PS2 / N64 revival with dual 64-bit RISC-V + full RVV.

8 Conclusion

The BTRON BX series is an invitation to treat classic media-processor architectures as living subjects of study. By re-expressing their peripheral ecosystems in clean Bluespec SystemVerilog, and by imposing a single scalable vector architecture (VEX) from BX-16 upward, the series offers a practical path from 8-bit MSX-class machines to 64-bit vector architectures whose software model remains coherent across the entire family. The four cores—bx-8, bx-16, bx-32 and bx-64—form one graduated programme united by methodology, by a progressive vector capability, and by respect for the original engineering.

Umbrella technical note prepared for professional consideration.
November 2026.